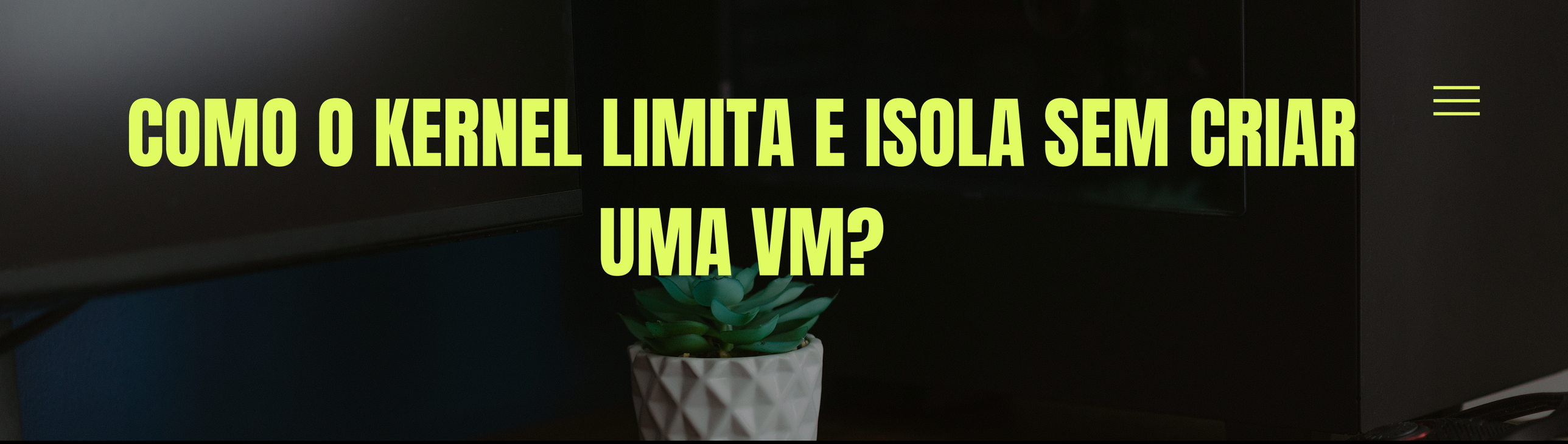


ISOLAMENTO E LIMITES DE RECURSOS EM CONTÊINERES LINUX

ANÁLISE EMPÍRICA DOS MECANISMOS DE NAMESPACES E CGROUPS V2 NO KERNEL LINUX 7.0 SEM HIPERVISORES INTERMEDIÁRIOS.

Docente: Glauco de Figueiredo Carneiro

Discentes: Beathriz Laurent, Samyr Teles, Rafael de Jesus, Luan Feitosa, Matheus de Oliveira, Evelyn Hilthegarden



COMO O KERNEL LIMITA E ISOLA SEM CRIAR UMA VM?

NAMESPACES

Particionamento de visão lógica. Define o que o processo consegue enxergar no sistema (PIDs, montagens, interfaces de rede).

CONTROL GROUPS (CGROUPS)

Medição e regulação de consumo. Define quanto de recurso físico o processo pode gastar (CPU, Memória RAM, PIDs).

Hipóteses de Trabalho (H1, H2, H3)

Três proposições fundamentais avaliadas empiricamente no laboratório.

H1

Isolamento de Visão via Namespaces

Contêineres da mesma imagem terão handles de namespace (PID, mnt, net) distintos. Ambas instâncias terão PID 1 interno, mapeados para PIDs sequenciais e diferentes no host.

H2

Gestão de Recursos via Cgroups

Para uma thread única fixada, a quota de 0,5 CPU (C1) exigirá o dobro do tempo de parede (wall time) da quota de 1,0 CPU (C2), exibindo throttling expressivo em `cpu.stat`.

H3

Proteção de Montagem Read-Only

Montagem de diretório com a flag `readonly` permitirá leitura e interceptará escrita diretamente no barramento de montagem, independente de permissões POSIX do host.



ESPECIFICAÇÕES DO AMBIENTE BARE-METAL

Hospedeiro: Linux nativo (Sem VM intermediária)

SO & Kernel: Ubuntu 24.04.2 LTS / Kernel 7.0.0-38-generic

Hardware (CPU): AMD Ryzen 5 5600GT (6 cores / 12 threads)

Memória / Swap: 15 GiB RAM (10 GiB livres) / 4,0 GiB Swap

Motor Docker: Docker Engine 29.1.3 / Compose 2.40.3

Hierarquia cgroup: cgroup v2 (driver systemd unificado)

Evidências Coletadas: Namespaces (PID, Mount, Net)

Comparativo dos handles extraídos via `docker inspect` e pseudonodos em `/proc/1/ns/`.

Recurso / Artefato	Contêiner A (av3-ns-a)	Contêiner B (av3-ns-b)	Resultado	Interpretação do Kernel
PID no Host (Inspect)	39466	39524	DIFERENTES	Processos distintos mapeados na tabela global do host.
PID 1 Interno (comm)	sleep	sleep	IGUAIS	Ilusão local de ser a tarefa inicial raiz do ambiente.
/proc/1/ns/pid	pid:[4026532892]	pid:[4026532964]	DIFERENTES	Instâncias isoladas de tabela de sinalização de processos.
/proc/1/ns/mnt	mnt:[4026532880]	mnt:[4026532961]	DIFERENTES	Árvores de diretórios e rootfs totalmente independentes.
/proc/1/ns/net	net:[4026532894]	net:[4026532966]	DIFERENTES	Pilhas lógicas de rede segregadas (`--network none`).

Ressalva Analítica: Os handles atestam a segregação de escopo, mas não provam segurança absoluta perimetral contra escapamentos de contêiner e nem impõem limitação de CPU/RAM.

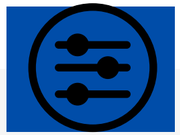
CARGA SINTÉTICA DETERMINÍSTICA (PBKDF2)

Esforço Calibrado



200 blocos de 100k loops encadeados em um checksum SHA-256 definitivo (6140a0e6...).

Condições de Quota



C1: 0,5 CPU (cpu.max=50000 100000)
C2: 1,0 CPU (cpu.max=100000 100000)

Travas de Segurança



RAM teto 256 MiB (swap=0), max 64 PIDs, cap-drop ALL e timeout externo de 90s.

Razão Escalar Obtida: 1,96×

Convergência quase linear com a expectativa teórica da quota fracionada (esperado: 2,00×).

5,201s Wall Mediano C1 (0,5 CPU)

2,650s Wall Mediano C2 (1,0 CPU)

Condição	Wall Mediano	CPU Processo	Taxa (it/s)	Throttled (`nr_throttled`)	Tempo Estrangulado	Pico RAM (cgroup)
C1 (0,5 CPU)	5,201 s	2,600 s	3,845M	52 / 52 períodos (100%)	2.599,1 ms	12,3 MiB
C2 (1,0 CPU)	2,650 s	2,567 s	7,547M	3 / 26 períodos (Residual)	1,1 ms	12,7 MiB

Conclusão do Cgroup: Em C1, o estrangulamento imposto pelo escalonador CFS atuou em 100% das janelas temporais de 100ms. Em C2, o tempo de CPU igualou-se ao wall time (`CPU/Wall = 0,999`).

Proteção de Montagem Read-Only

Intercepção de escrita diretamente no nível do barramento de montagem do kernel Linux.

```
// Registros brutos obtidos no container em tempo real

===== TESTE DE LEITURA =====
Documento sintetico da AV3

===== TESTE DE ESCRITA =====
sh: 6: cannot create /dados/tentativa.txt: Read-only file system
SUCESSO: escrita negada
```

Negação no Barramento

A mensagem `Read-only file system` comprova a bloqueio pelo mnt namespace, diferente de `Permission denied` (POSIX).

Desvinculação de Permissão Host

A flag `readonly` na montagem atua proativamente antes da verificação de proprietário ou atributos CHMOD do arquivo.

FATORES DE OSCILAÇÃO E FREQUÊNCIA DE CPU

CPU FREQUENCY BOOST

O governador de energia do kernel operava dinamicamente entre 421 MHz e 4.668 MHz durante a coleta, introduzindo ruído microarquitetural.

DEGRADAÇÃO SEQUENCIAL

A 3ª repetição em ambas condições foi consistentemente mais lenta (+2,8% em C1 e +3,4% em C2), indicando aquecimento ou disputas no host.

CONCORRÊNCIA MULTIMÍDIA

Ambiente desktop ativo com 5,1 GiB de RAM ocupados por processos da interface gráfica e fundo no momento do benchmark.

AFINIDADE NÃO FIXADA

Cgroups em C1 limitaram tempo de quota CFS agregada, sem fixação rígida de núcleo (cpuset não foi restringido).

Importância dos Limites em Provedores de Inferência

Mitigação de riscos críticos em clusters de produção servindo modelos generativos.

Prevenção do OOM Killer

Sem memory.max, um vazamento de memória num pipeline RAG faria o kernel encerrar arbitrariamente contêineres vizinhos do cluster.

Proteção contra Poisoning

Montar a base documental vetorial como readonly impede adulteração do corpus de conhecimento mesmo em explorações RCE.

Vazamento de VRAM em GPUs

Cgroups nativos de Linux não regulam VRAM de placas discretas. Limites exigem ferramentas de terceiros (ex: NVIDIA Container Toolkit).

Protocolos de Contingência do Lab

- **Absolvição de Ataques Destrutivos:** Não foram executados Fork Bombs ou indução intencional de OOM.
- **Timeouts em Camadas:** Limite algorítmico interno de 60s + GNU Timeout externo de 90s no host.
- **Sem Escrita em /sys:** Preservação integral do agendador e parâmetros globais do host.

Uso Crítico e Declarado de IA

Modelo Utilizado: Claude Sonnet 5.0.

Escopo de Atuação: Suporte no refatoramento de scripts Python herdados da Atividade 2 para coleta automatizada de CSV/JSON e tratamento sintático. Todos os dados brutos e execuções foram auditados rigorosamente pela equipe.

CONCLUSÕES DO LABORATÓRIO

A experimentação provou empiricamente que contêineres dependem da sinergia entre Namespaces (isolamento de visão) e Cgroups (regulação de orçamento), operando diretamente no kernel hospedeiro sem sobrecarga de virtualização de hardware.

A ausência de hypervisors garante acesso direto ao hardware, onde o kernel único gerencia preempção e chamadas de sistema sem a sobrecarga de VMs.

Namespaces isolam a visão (PID, mnt, net) e Cgroups regulam a quota (CPU/RAM/PIDs). Isolamento sem cgroups falharia sob picos de estresse.

Foi imprescindível em arquiteturas RAG/LLM para conter vazamentos via OOM e blindar repositórios (mounts ro), ressalvando limitações de VRAM em GPUs.